

PromptABI: Static Verification for the Discrete Interface Layer of LLM Systems

PromptABI Contributors

Abstract

Modern LLM systems fail in places that are not neural: chat templates splice untrusted text into role delimiters, tokenizer metadata changes the strings that grammars actually see, stop sequences fire inside tool arguments, provider adapters reinterpret structured outputs, and framework truncation silently drops the evidence a prompt was meant to preserve. These failures are discrete, versioned, and usually present before a single token is generated, yet they are typically discovered through inference-time tests or production incidents.

PromptABI treats this boundary as an application binary interface for prompts. It gives templates, tokenizers, stop policies, tool schemas, provider envelopes, retrieval budgets, training manifests, and evaluation harnesses a common typed semantics. It then runs bounded symbolic execution, finite-language products, differential replay, and SMT-backed finite contracts to prove one of four useful facts: the interface is safe in the supported fragment, a concrete counterexample exists, the checker must abstain, or a versioned artifact has drifted from a previously verified contract.

The result is a CPU-only verifier that can be run in CI, before fine-tuning, before publishing benchmark scores, or before migrating providers. Its evidence is not a checklist of repository files: diagnostics carry executable witnesses, source spans, token paths, parser states, solver models, replay hashes, and privacy-preserving fingerprints. The reported bug-finding evaluation uses public bug replays that traverse exact pinned production-code excerpts.

1 Introduction

The software around an LLM has become a compiler pipeline. A user message is rendered by a chat template, transformed by tokenizer normalization and special token rules, packed into a context window, constrained by a grammar or JSON schema, streamed through stop policies, decoded by a provider adapter, and possibly reused as training or evaluation data. The model may be probabilistic, but this surrounding pipeline is largely deterministic. When it is wrong, the failure is often predictable without running the model.

PromptABI is built on that observation. It verifies the discrete interface layer of LLM systems before inference. The name is intentional: like an ABI for native code, the prompt interface defines what representations cross subsystem boundaries and which invariants downstream components assume. If a prompt template says that user content is data, but renders raw role delimiters inside that content, the ABI is broken. If a tool parser assumes that `</tool_call>` is structural, but a valid string argument may contain that sequence, the ABI is broken. If an evaluation harness scores a model after dropping the system prompt under a framework’s packing policy, the ABI is broken.

This paper presents PromptABI as a programming-languages tool for LLM applications. The focus is not model quality, prompt style, or policy moderation. It is the statically checkable semantics of the artifacts that connect a model to the rest of a program.

Contributions. PromptABI contributes:

1. a typed interface model for prompt-system artifacts that are otherwise checked independently or only by example;
2. bounded semantics for chat-template rendering, tokenization, stop reachability, structured outputs, provider migration, context budgets, training masks, and evaluation harnesses;
3. proof-producing diagnostics that distinguish verified safety, concrete counterexamples, and honest abstention;
4. a deterministic CLI and embedding API designed for CI and offline use;
5. an evaluation suite whose reported bug-finding results come from public production code in the wild, not synthetic simplifications of those bugs.

2 The Need

LLM applications are commonly tested end-to-end: run a model, inspect a sample, and patch the prompt when behavior looks wrong. That style misses interface bugs for three reasons.

The bug may be rare under sampling but inevitable in the interface. If a stop string is allowed inside a valid tool argument, generation may or may not produce it in a test run. The interface, however, admits the bad string regardless of the model distribution.

The responsible component is often not the model. Many incidents are caused by a tokenizer revision, chat-template update, provider adapter, streaming parser, framework truncation policy, or evaluation wrapper. These artifacts are versioned software inputs and can be checked like software inputs.

A passing run is not a proof. An inference test demonstrates one execution. A verifier can instead enumerate the bounded states of the interface, produce a minimal witness, and record why a claim is valid only in a supported fragment.

PromptABI addresses these gaps by moving checks left: the same artifacts that would be passed to inference are loaded, normalized, and checked before any weights or provider credentials are needed.

3 A Prompt ABI

Definition 1 (Prompt interface artifact). *A prompt interface artifact is a discrete, versioned object that influences the string or structured value seen by an LLM subsystem. Examples include chat templates, tokenizer configs, stop policies, JSON schemas, tool definitions, provider request/response envelopes, truncation policies, prompt segments, training manifests, and evaluation manifests.*

Definition 2 (Prompt ABI). *A prompt ABI is the finite contract induced by a set of prompt interface artifacts: which roles may appear, which strings are data versus control, which tokens are reserved, which regions must survive packing, which grammars can be accepted, which stops may fire, and which structured fields downstream parsers expect.*

PromptABI compiles each artifact into small semantic views. A chat template becomes a bounded set of symbolic render paths and role regions. A tokenizer becomes an encode/normalize/decode relation over finite witnesses. A JSON schema becomes a grammar fragment and representative values. A provider adapter becomes a contract over fields, tool-call encodings, streaming chunks, stops, and structured-output modes. A training or evaluation manifest becomes a set of finite obligations about roles, masks, preserved context, answer schemas, and private fields.

The tool deliberately does not claim to model arbitrary Python, arbitrary Jinja, or arbitrary parser code. Its supported fragments are explicit. Outside them, PromptABI reports abstention instead of returning a false proof.

4 How PromptABI Checks Interfaces

PromptABI’s checkers share a common pattern:

1. load typed artifacts with source spans, hashes, and version metadata;
2. compile them to bounded semantic objects;
3. run a product, symbolic execution, differential replay, or SMT query;
4. emit either a proof summary, a concrete witness, or an abstention.

4.1 Role Boundaries

The role-boundary checker symbolically executes supported chat-template fragments. It partitions each rendered path into structural regions such as system, user, assistant, tool, developer, and function. It then asks whether an attacker-controlled field can render a structural marker: role headers, special tokens, assistant prefixes, tool delimiters, markdown fences, or provider sentinels.

For example, a template that renders

```
'<|user|>\n' + message['content'] + '<|end|>\n'
```

without escaping permits user content to contain `<|system|>` or `<|end|>` as literal control text. PromptABI reports the input field, the marker, the rendered excerpt, the tokenized representation, and the exact offset of the forged boundary.

4.2 Stops and Structured Outputs

Stop policies are safe only relative to the language of outputs they interrupt. PromptABI builds bounded structured-output regions for JSON strings, tool arguments, XML-like tool calls, markdown fences, and provider tool-call envelopes. It then checks whether a configured stop sequence can occur before a valid structured output is complete.

The witness records the stop, the valid full output, the prefix returned after truncation, the parser state at the firing point, and the malformed or prematurely accepted structure a downstream parser would receive.

4.3 Tokenizers and Grammars

Tokenizer bugs often arise from mismatches between text-level constraints and token-level behavior. PromptABI can compare tokenizer revisions, detect drift in special-token IDs and normalization, and form bounded products between a tokenizer relation and a grammar. This exposes empty products, ambiguous token paths, decoded-text conflicts, whitespace aliases, Unicode normalization surprises, and added-token interactions.

4.4 Budgets, Training, and Evaluation

Context windows are treated as finite packing problems. PromptABI combines declared segments, tokenizer-derived or declared token counts, output reserves, tool overhead, generation prompts, and framework truncation policies to prove whether required regions survive. The same machinery applies to RAG citations, training masks, supervised target roles, DPO/RLHF pair consistency, and evaluation harness prompts. A benchmark score should not be trusted if the grader rubric, answer schema, system prompt, or private answer key can be dropped or exposed by the interface.

4.5 Provider Migration

Provider migration is checked as a contract comparison, not as string matching. PromptABI compares request and response surfaces: tool-call IDs, JSON-object versus JSON-string arguments, parallel-call support, streaming chunk assembly, stop semantics, context limits, structured-output modes, and error envelopes. The output is a set of migration diagnostics with concrete fields and suggested safe fixes.

5 What the Tool Can Do

Table 1 summarizes the main classes of checks. The table is organized by programmer need rather than by implementation module.

Need	PromptABI check	Evidence produced
Prevent prompt control injection	Role-boundary non-forgability over bounded template paths	Forged marker, input field, rendered region, token evidence
Avoid premature parser truncation	Stop-overreachability over schemas, tools, and provider envelopes	Stop firing point, parser state, malformed prefix
Trust constrained decoding	Tokenizer $\times$ grammar emptiness and ambiguity checks	Token path, decoded text, grammar state, minimal string
Migrate providers safely	Provider and streaming contract comparison	Lost field, changed encoding, response-shape witness
Protect RAG, training, and eval invariants	Context-budget, mask, role, and private-field obligations	Survival proof, dropped-region witness, finite assignment
Maintain reproducible CI	Lockfiles, signed bundles, replayable solver certificates	Artifact hash, diagnostic fingerprint, replay hash

Table 1: PromptABI capabilities stated as user-facing verification needs.

6 How It Proves Things

PromptABI uses proof techniques that match the artifact being checked. It does not use one universal solver for every problem.

Bounded symbolic execution. For supported chat-template fragments, PromptABI enumerates bounded control-flow paths and loop counts. Each path is a sequence of symbolic segments annotated with role ownership and source expressions. A finding is constructed only when a user-controlled expression is in a region whose control markers are known.

Finite-language products. Tokenizers, grammars, and parser fragments are compiled to finite relations for the witnesses under consideration. Products and projections answer emptiness, ambiguity, and reachability questions with concrete paths.

SMT-backed finite contracts. Static contracts that combine budgets, roles, stops, masks, and provider facts are lowered to finite SMT obligations. Solver models become counterexamples; unsatisfiable obligations become proof summaries. Reduced solver-replay files allow a result to be rerun without the original private artifacts.

Differential replay. Where a library is the specification, PromptABI runs small differential cases against real tokenizer/template/parser behavior. This catches divergence between the local abstraction and the implementation developers deploy.

Fail-closed abstention. If a construct is outside the supported fragment, a dependency is missing, a schema cannot be modeled, or a source-derived value cannot be recovered, PromptABI reports that fact. It does not silently turn an unsupported case into a safe result.

Theorem 1 (Bounded witness soundness). *For a supported checker fragment, every unsafe PromptABI diagnostic contains a witness that is accepted by the artifact semantics used by that checker and violates the stated interface obligation.*

Proof sketch. Each checker constructs diagnostics from semantic objects, not from ad-hoc strings. The template checker builds a witness from an enumerated symbolic path, role region, and input expression. The stop checker substitutes the stop into a valid structured-output witness and records the parser state at truncation. The grammar/tokenizer check records a concrete token path through the product. The SMT layer records the satisfying assignment or unsat result for the lowered finite obligation. Tests replay these witnesses through the same checker semantics and, where applicable, real library adapters. $\square$

Theorem 2 (Honest fragment boundary). *PromptABI’s successful safety claims are scoped to explicit supported fragments; inputs outside those fragments produce abstentions or setup failures rather than passing diagnostics.*

Proof sketch. Artifact loaders and checkers validate dialects, parser support, dependency availability, schema fragments, and source-derived replay assumptions before claiming success. Unsupported constructs are recorded in the diagnostic model, and release tests require abstentions to remain visible. $\square$

7 Implementation

PromptABI is a Python tool with a deterministic command-line interface and an embedding API. The implementation is organized around typed artifacts and typed diagnostics rather than around particular frameworks. The same verification result can be rendered as text, JSON, SARIF, GitHub annotations, HTML reports, signed bundles, or downstream integration payloads.

The CLI is intentionally CPU-only for the checks described here. That property is important: users can run PromptABI in pull requests, air-gapped builds, pre-commit hooks, release gates, fine-tuning preflight jobs, benchmark publication workflows, and provider migration reviews without model weights or provider credentials.

Privacy is part of the implementation contract. Diagnostics can expose exact structural witnesses for public fixtures, but private workflows can choose position-preserving redaction, hash-only evidence, or structural summaries while preserving stable fingerprints.

8 Evaluation

PromptABI’s evaluation separates verifier-maintenance tests from real-world claims. Unit tests, property tests, and small local fixtures are useful for maintaining the implementation, but they are not counted here as bug-finding evidence. The evaluation in this paper counts only public upstream code that was shipped in the wild and is pinned by repository, commit, path, license, and SHA-256 hash.

8.1 Research Questions

The evaluation asks four questions.

1. **Bug finding on real code:** when given exact production artifacts from public incidents, does PromptABI find the interface bug?
2. **Attribution:** does the diagnostic identify the production artifact and boundary responsible for the bug?
3. **Replayability:** can the result be rerun offline from pinned public source bytes without model weights, provider calls, or private data?
4. **Cost:** is the check cheap enough for CI?

8.2 GitHub Bug Search and Corpus

We searched public GitHub issues and pull requests in llama.cpp, vLLM, and Hugging Face Transformers for recorded failures involving chat templates, role delimiters, tool parsers, stop strings, quote sentinels, streaming parser state, and structured-output boundaries. The search identified ten PromptABI-class records. A record was admitted to the evaluation only if the issue or PR named an upstream production artifact and PromptABI could recover the relevant boundary from exact pinned source bytes. Six records meet that bar in the current corpus. Records that require hand-reduced payloads, live model sampling, or a verifier family not yet implemented are excluded from the counted evaluation.

Table 2 lists the current evaluation corpus. Each row is exact code in the wild. The tool loads the pinned source excerpt, validates the source hash, extracts the template, parser delimiter, parser regex, or parser control-flow pattern from those bytes, and only then runs the checker. If the expected value cannot be recovered from the source, the replay fails.

8.3 Results

PromptABI agrees with all six GitHub-recorded bugs in the corpus. The Gemma4 quote case finds the exact production parser pattern described in vLLM issue #39069: `_ESCAPE_TOKEN` is replaced with a raw quote, then fallback parsing uses a regex fragment that stops at the first internal quote. The Gemma4 streaming case finds the source pattern from issue #38910 and PR #38909: buffered delta text is reconstructed into the parser’s accumulated input. The llama.cpp `TAG_WITH_TAGGED` case finds the source branch described in issue #21771, where non-string tagged parameters enter `p.json()` inside the tag grammar. The Phi case yields 18 role-boundary witnesses after traversing 430 bytes of the exact upstream template. The Qwen3 Coder case yields 2 stop-overreach witnesses after extracting `</tool_call>`, `</function>`, and `</parameter>` from exact upstream parser source. The Qwen3 XML case finds the stale function-state pattern from issue #43713. On a local CPU run, the complete evaluation command finishes in 0.61 seconds wall-clock time.

8.4 Interpretation

The evaluation is intentionally selective rather than padded. The search found more recorded PromptABI-class failures than are counted here, but we did not add cases merely because an issue contained a convincing payload. Non-production regression fixtures are excluded from the evaluation claim. The claim is: for the counted cases, PromptABI traverses the same public code that shipped upstream, recovers the relevant boundary from that code, and produces concrete diagnostics before inference.

Production artifact	Public incident	Property checked	Result
vLLM <code>gemma4_utils.py</code> at commit 6a11d72	Issue #39069: internal quotes truncate Gemma4 tool arguments	Does source replace the Gemma quote sentinel with " and then parse with a quote-stopping fallback regex?	Bug found: parser truncation pattern
vLLM <code>gemma4_tool_parser.py</code> before PR #38909	Issue #38910: streamed HTML tags duplicated inside tool arguments	Does streaming buffer output get stitched back into the accumulated parser input?	Bug found: buffer reparse pattern
llama.cpp <code>chat-auto-parser-generator.cpp</code> at commit 2016bf2	Issue #21771: Qwen3 <code>TOPIC_WITH_TAGGED</code> array-of-object arguments poison streaming history	Does the non-string tagged-parameter branch route structured arguments through <code>p.json()</code> inside the tag grammar?	Bug found: JSON-parameter boundary pattern
llama.cpp <code>microsoft-Phi-3.5-mini-instruct-janitor</code> at commit 2016bf2	PR #18462: Phi-style role delimiters jangling	Can raw message content forge role/special-token boundaries?	Bug found: 18 role-boundary witnesses
vLLM <code>qwen3coder_tool_parser.py</code> at commit 6a11d72	PR #40861: Qwen3 XML tool parser regressions	Can parser stop delimiters occur inside a valid tool argument boundary?	Bug found: 2 stop-overreach witnesses
vLLM <code>qwen3xml_tool_parser.py</code> at commit 6a11d72	Issue #43713: multiple <code><function=...></code> blocks emit invalid JSON	Does function-open auto-close reuse stale function state without a fresh tool-call boundary?	Bug found: stale function-state pattern

Table 2: Evaluation cases. Every case is replayed from exact pinned upstream production code; synthetic reductions are not counted.

Metric	Value
Exact upstream production-code cases	6
Cases with expected bug found	6
Abstentions in counted cases	0
Model or provider calls	0
Total concrete witnesses/pattern agreements	30
Wall-clock runtime	0.61s

Table 3: Current production-code evaluation results.

9 Limitations and Non-Goals

PromptABI is not a model-behavior verifier. It does not prove that an LLM will answer truthfully, refuse harmful requests, or choose a tool wisely. It proves properties of the discrete interface that surrounds the model.

PromptABI is also intentionally bounded. Chat-template loops, grammar products, schema recursion, SMT domains, and context-budget facts are finite. That makes diagnostics replayable and CI-friendly, but it means claims must be read with their bounds and supported-fragment metadata.

Finally, PromptABI is not a sandbox. It avoids provider calls and model execution for verification, and it can operate on redacted evidence, but process isolation and secret governance remain deployment responsibilities.

10 Related Work

PromptABI sits between several families of tools. Prompt linters and jailbreak tests look for risky strings or behavioral failures, but they do not generally compile tokenizer/template/provider artifacts into a shared semantics. Schema validators and constrained decoders help one structured-output layer, but they usually do not account for tokenizer drift, stop policies, provider envelopes, or downstream parsers. Formal methods tools can prove finite obligations, but they are rarely packaged around the artifacts LLM engineers already maintain. PromptABI’s contribution is to bring PL-style static and differential reasoning to those concrete artifacts.

11 Conclusion

LLM systems need a verifier for the deterministic software boundary around the model. PromptABI supplies that verifier. It treats prompt templates, tokenizers, stops, schemas, tools, providers, budgets, training manifests, and evaluation harnesses as a prompt ABI; compiles them to bounded semantics; and emits replayable proofs, counterexamples, or abstentions before inference.

The resulting workflow is practical because it is CPU-only, version-aware, source-mapped, privacy-conscious, and integrated with CI. It is credible because its diagnostics carry executable witnesses and because its real-bug evaluation traverses exact production code that shipped in the wild.